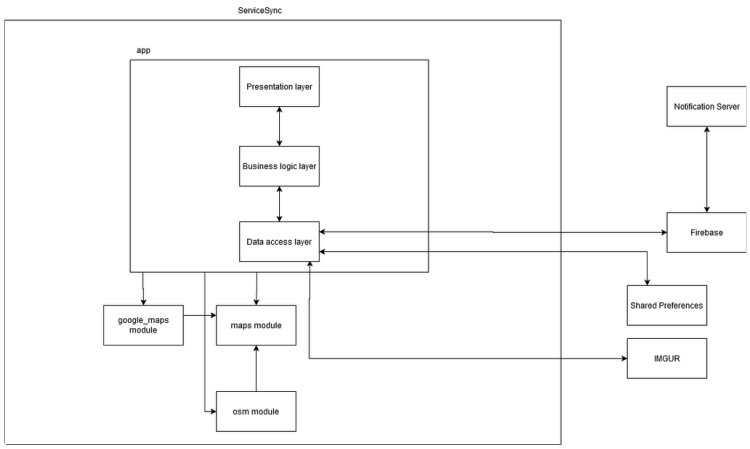
Architecture

1. Architecture Overview

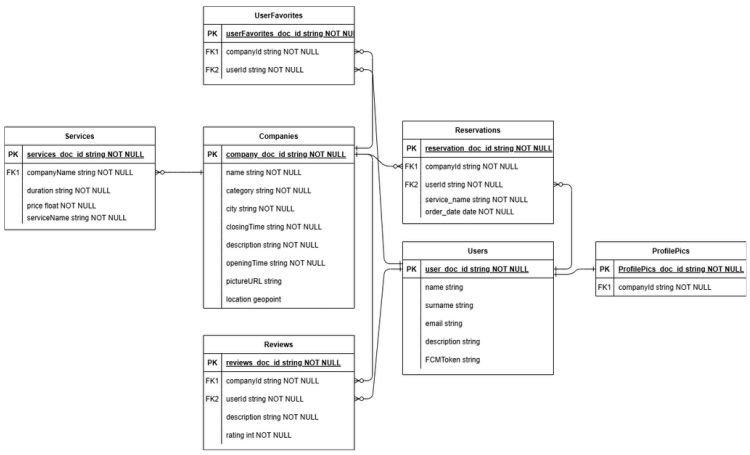
This application is built using a three-layered architecture that provides clear separation of concerns, enhancing code maintainability and scalability. The architecture is organized into three main layers: **Presentation Layer**, **Business Logic Layer**, and **Data Access Layer**. Each layer is designed to handle specific responsibilities, ensuring modularity and simplifying further development.

2. Architecture Diagram



ServiceSync General Architecture

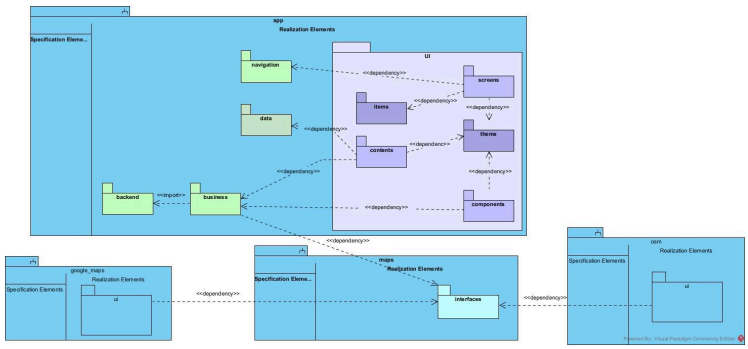
3. ERA Diagram



ERA diagram of the project. Note - it uses FireStore so it does not have strict tables as such, though it is still recommended to be used as if it is one in this project.

4. Package diagram of the system

Because there is a lot of packages, and many of them have a few dependencies, this diagram shows more details about how the integrity of ServiceSync works.

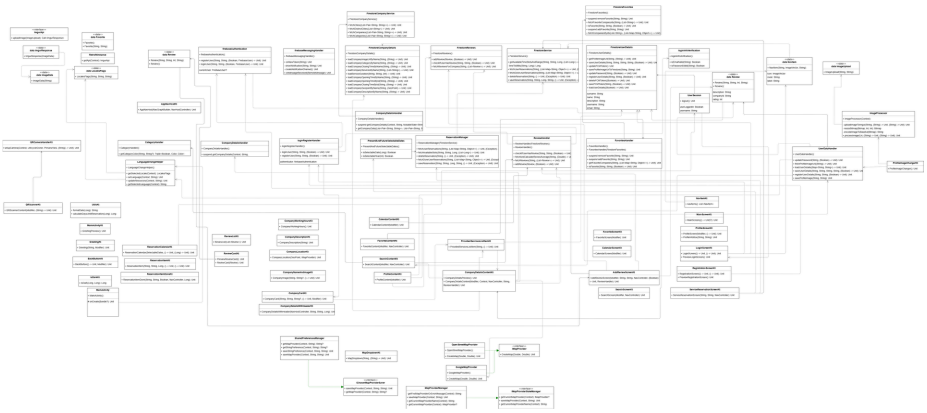


ServiceSync Package Diagram

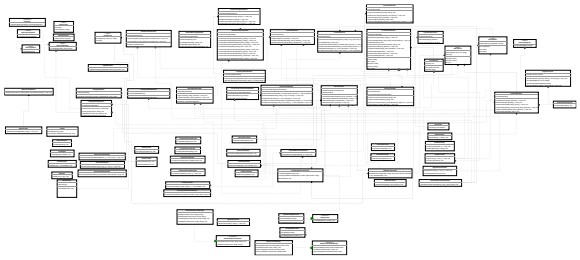
The general usage is mostly in **servicesync** package, but the implementation of our maps and providers are located separately in **maps**, for easier access and better modularity.

5. Class Diagram

Complete class diagram arranged in a way that tree layers are visible on the diagram. On the bottom, the “maps” module is noticeable too. Compatibility-wise, .png version is also included right below.



Dijagram klasa (PNG)



Kompletan dijagram klasa (SVG)

6. Layer Descriptions

Layer	Description	Main Responsibilities
Presentation Layer	This layer is responsible for displaying data to the user and handling all user interactions. It interacts with the Business Logic Layer to present updated data and state changes.	UI rendering, user interaction management
Business Logic Layer	Acts as the core layer, managing data flow between the Presentation Layer and the Data Access Layer. It applies business rules and prepares data for display.	Data processing, business rules, and application logic
Data Access Layer	Handles all data-related operations, including data retrieval, storage, and communication with external APIs or databases. This layer is responsible for managing data sources independently of other layers.	Data access, storage management, API/database handling

7. Data Flow and Component Interactions

Data in this three-layered architecture flows as follows:

1. **Data Access Layer**: Responsible for retrieving data from external sources (APIs, databases) and providing it to the Business Logic Layer. This layer is decoupled from the Presentation Layer to ensure flexibility in data handling.
2. **Business Logic Layer**: Processes and prepares data for the Presentation Layer. It applies necessary business rules, transformations, or calculations, ensuring that data sent to the UI is in the required format.
3. **Presentation Layer**: Displays the processed data and handles user interactions, such as button clicks or form inputs. It sends any user actions to the Business Logic Layer, which then manages any required data updates or additional processing.

8. Key Responsibilities of Each Layer

- **Presentation Layer**: Manages the display and interaction of data with the user, updating the interface based on the latest data from the Business Logic Layer.
- **Business Logic Layer**: Acts as a mediator between Presentation and Data Access Layers, applying business rules and data transformations.
- **Data Access Layer**: Handles direct access to data sources, ensuring that data retrieval and storage are performed independently of other layers.